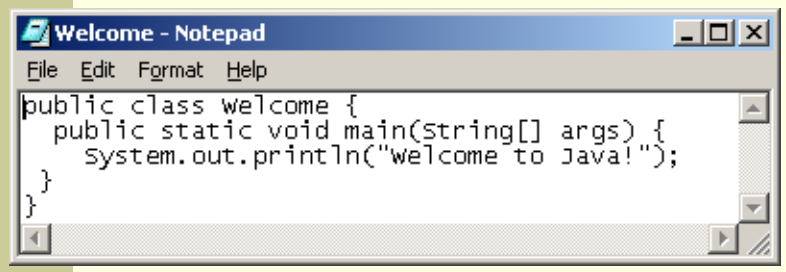
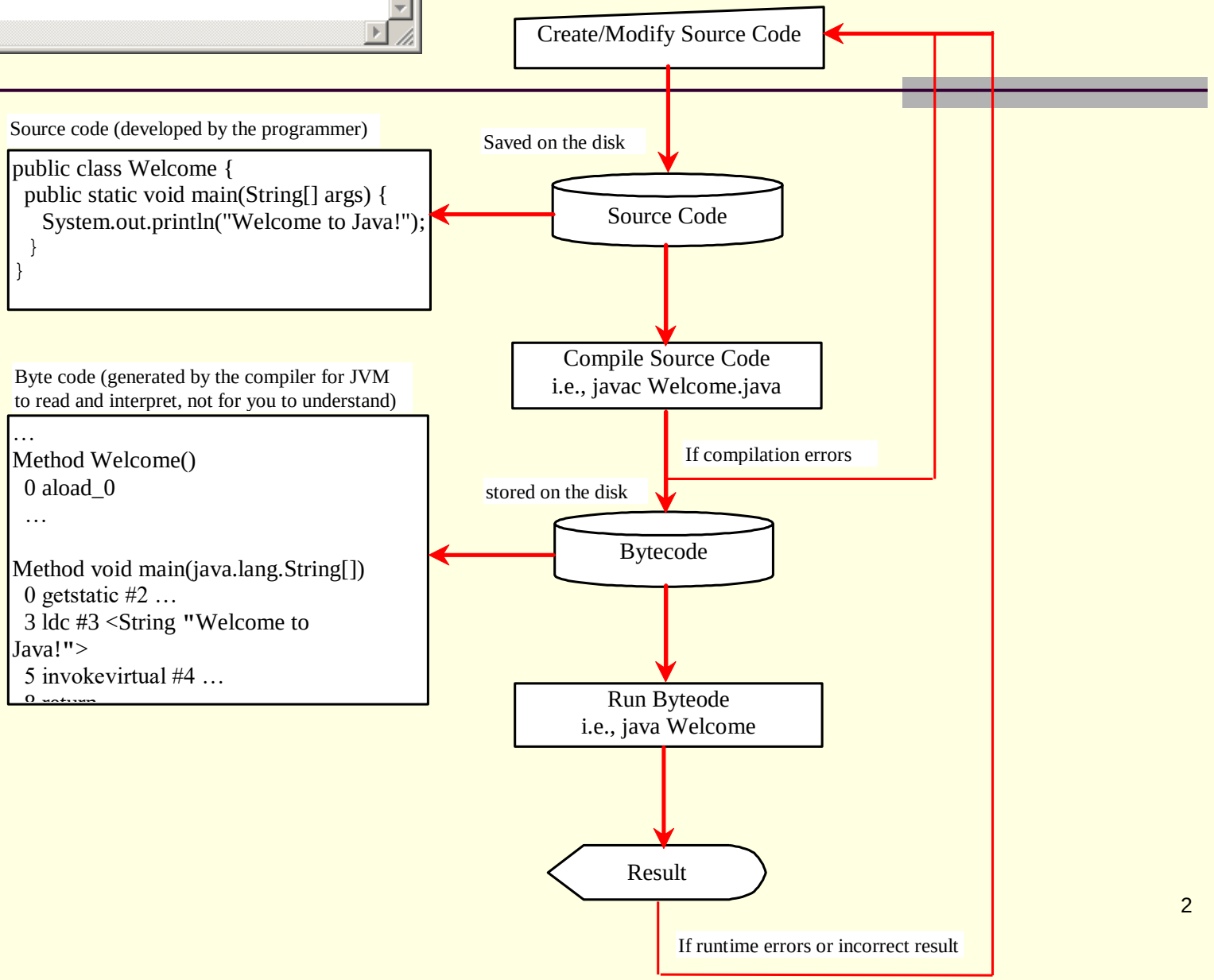
Anatomy of a Java Program

Lecture 3

Based on Slides of Dr. Norazah Yusof

Creating, Compiling, and Running Programs





Parts of a Java Program

- A Java source code file contains one or more Java classes.
- If more than one class is in a source code file, only one of them may be public.
- The public class and the filename of the source code file must match.

  ex: A class named `HelloApp` must be in a file named `HelloApp.java`
- Each Java class can be separated into parts.

Parts of a Java Program

- Example: HelloApp.java
- To compile the example:
  - **`javac HelloApp.java`**
    - Notice the `.java` file extension is needed.
    - This will result in a file named `HelloApp.class` being created.
- To run the example:
  - **`java HelloApp`**
    - Notice there is no file extension here.
    - The *java* command assumes the extension is `.class`.

Analyzing the Example

```
// This is my first Java program.

public class HelloApp
{



}
```

This is a Java comment. It is ignored by the compiler.

This is the class header for the class HelloApp

This area is the body of the class HelloApp. All of the data and methods for this class will be between these curly braces.

Analyzing the Example

```
// This is my first Java program.

public class HelloApp
{
    public static void main(String [] args)
    {


    }

}
```

This is Java main method. Every Java application must have a main method.

This area is the body of the main method. All of the actions to be completed during the main method will be between these curly braces.

Array of string to pass inputs to the program via command line

Analyzing the Example

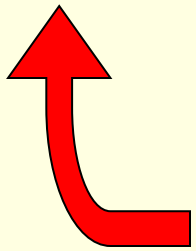
```
// This is my first Java program.

public class HelloApp
{
    public static void main(String [] args)
    {

      System.out.println("Programming is great fun!");

    }

}
```



This is the Java Statement that is executed when the program runs.

Anatomy of a Java Program

- Comments
- Keywords
- Modifiers
- Statements
- Blocks
- Classes
- Methods
- The main method
- Package

Comments

- Help the programmers to communicate and understand the program.
- Not a programming statement, thus ignored by the compiler.
- Preceded with // on a line
- Enclosed between /* and */ on one or several lines.

Comments on several lines

```
1.  /*
2.   *   This is the first sample program in Java Programming.
3.   *   Copyright (C) 2000 Norazah Yusof
4.   */
5.
6.  public class FirstSample
7.  {
8.      public static void main(String[] args)
9.      {
10.       System.out.println("Selamat Tahun Baru");
11.     }
12. }
```

Key Words

- Words that have a specific meaning to the compiler
- Examples:

•public	•void	•boolean	•private
•class	•int	•continue	•protected
•static	•double	•return	•package

- Key words are lower case (Java is a case sensitive language).
- Key words cannot be used as a programmer-defined identifier.

Java Reserved Keywords

Category	Keyword	Description
Access Modifiers	`public`	Makes a class, method, or variable accessible everywhere.
	`private`	Restricts access to within the same class.
	`protected`	Allows access within the same package and subclasses.
	`default` (implicit)	Package-private access (when no modifier is specified).
Data Types	`int`	Integer data type.
	`double`	Double-precision floating-point number.
	`float`	Single-precision floating-point number.
	`char`	Character data type.
	`boolean`	Represents `true` or `false` values.
	`byte`, `short`, `long`	Other numeric data types.

Java Reserved Keywords

Class & Object Handling	`class`	Defines a class.
	`interface`	Defines an interface.
	`extends`	Inherits a class (used for inheritance).
	`implements`	Implements an interface.
	`this`	Refers to the current object.
	`super`	Refers to the parent class.
	`new`	Creates new objects.
Control Flow	`if`, `else`	Conditional statements.
	`switch`, `case`	Switch-case branching.
	`while`, `do-while`	Looping structures.
	`for`	For loop.
	`break`	Exits a loop or switch.
	`continue`	Skips to the next loop iteration.

Java Reserved Keywords

Exception Handling	`try` , `catch`	Handles exceptions.
	`finally`	Block that always executes after `try-catch` .
	`throw`	Throws an exception.
	`throws`	Declares exceptions in a method signature.
Modifiers	`static`	Defines a static member (belongs to the class, not instances).
	`final`	Prevents modification (constant, method override prevention, etc.).
	`abstract`	Defines an abstract class or method.
	`synchronized`	Ensures thread-safe execution.
	`volatile`	Ensures visibility in multithreading.

Java Reserved Keywords

Packages & Imports	`package`	Defines a package.
	`import`	Imports classes from other packages.
Others	`return`	Returns a value from a method.
	`void`	Specifies no return type.
	`instanceof`	Checks object type at runtime.
	`enum`	Defines an enumeration.
	`native`	Specifies a method implemented in native code (C/C++).
	`transient`	Prevents serialization of a field.
	`strictfp`	Enforces strict floating-point calculations.
	`assert`	Used for debugging (assertions).

Basic Class Implementation

- Example:

```
public class ClassA {
    public static void main (String[] args) {
      System.out.println ("Try your best");
    }
}
```

Identify Java's reserved keywords in this example:

1. ???
2. ???

…

Statements

- Represents an action or a sequence of actions.
- Example of statement:

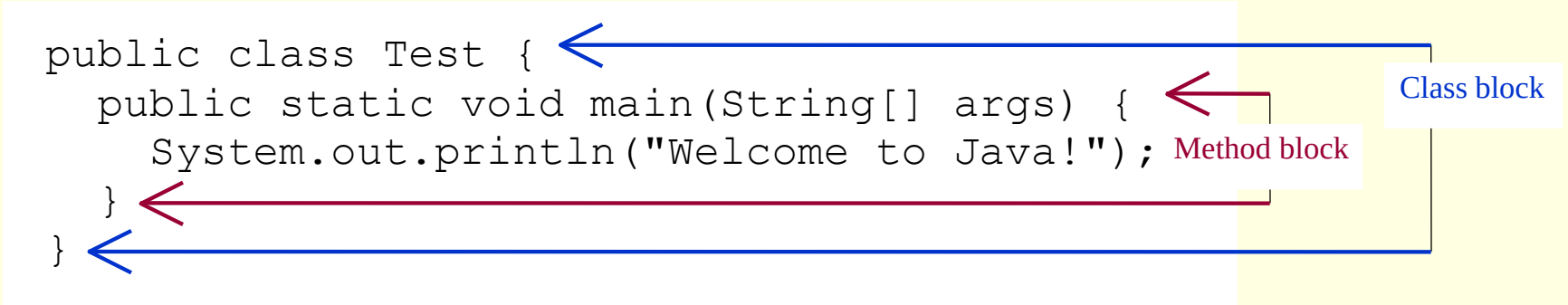
  `System.out.println("Welcome to Java!")`

  is a statement to display the greeting

  `"Welcome to Java!"`
- Every statement in Java ends with a semicolon (;).

Blocks

- Groups the components of the program using the braces `{` and `}` in the program.
- Every class has a *class block* that groups the data and the methods of the class.
- Every method has a *method block* that groups the data and the methods of the class.
- Block may be nested, meaning that one block can be placed within another.



Classes

- **class** is the essential Java keyword since classes are central to Java

- Programming in Java consists of defining a number of classes:
  - Every program is a class (A program is defined by using one or more classes.)
  - All programmer-defined types are classes

Classes

- Example 1:

```
public class ClassA {
    public static void main (String[] args) {
      System.out.println ("Try your best");
    }
}
```

Classes

- Example 2: Program named ClassA.java below has two classes i.e. ClassA and ClassB.

```
public class ClassA {
  private int yearborn=1988;
  public String methodA() { return "Aim High";  }
  public int getYearBorn() { return yearborn; }
}

class ClassB {
    public static void main (String[] args) {
      ClassA obj1 = new ClassA();
      System.out.println ("Your age: " + (2009 - obj1.getYearBorn()));
      System.out.println ("My message: " + obj1.methodA());
    }
}
```

Methods

- A collection of statements that performs a sequence of operations contained in a class.

- If a method is intended to be used to communicate with or pass information to an object, it should be declared `public`.

- Example: Method **`println()`** is an **instance method** that belongs to an object instance and is applied to an object (**`System.out`**).

Methods

getYearBorn **is a class method in** ClassA**.** public **modifier indicates it can be accessed from anywhere.** int **indicates it return a value of type integer.**

- Example:

```
public class ClassA {
  private int yearborn=1988;
  public String methodA() { return "Aim High";  }
  public int getYearBorn() { return yearborn; }
}

class ClassB {
    public static void main (String[] args) {
      ClassA obj1 = new ClassA();
      System.out.println ("Your age: " + (2009 - obj1.getYearBorn()));
      System.out.println ("My message: " + obj1.methodA());
    }
}
```

methodA **is a class method in** ClassA**.** public **modifier indicates it can be accessed from anywhere.** String **indicates it return a value of String.**

main **method is in** ClassB

getYearBorn **method is invoked from instance of the class,** obj1**.**

methodA **method is invoked from instance of the class,** obj1**.**

main Method

This is the parameter of the `main` method. It takes arguments of an array of Strings. The data type `String` starts with an upper-case `S`. The square brackets indicate an array.

- Every Java application must have a `main` method that is declared in the following way:

Keyword `void` indicates the data type returned from this method is nothing or no value.

```
public class ClassName
{
   public static void main(String[] args) {
      // Statements;
   }
}
```

The `main` method in Java is always `static`, meaning that this method can be run without creating an instance of the class.

This method is `public`, i.e. visible from anywhere that can see this class.

Command-Line Arguments

```
C:\norazah> javac Greetings.java
C:\norazah> java Greetings Aqilah Ahmad

Hello, Aqilah Ahmad
```

```
public class Greetings
{
    public static void main(String[] args)
    {
        String firstName = args[ 0 ];
        String lastName = args[ 1 ];
        System.out.println("Hello, " + firstName + "  " + lastName);
    }
}
```

Command-line arguments are passed to **main** as an array of **String**s.

Libraries

- Java programs are usually not written from scratch.
- There are hundreds of library classes for all occasions.
- Library classes are organized into packages. For example:

  java.util — miscellaneous utility classes

  java.awt — windowing and graphics toolkit

  javax.swing — GUI development package

import

- Full library class names include the package name. For example:

  java.awt. Color

  javax.swing.JButton

- import statements at the top of the source file let you refer to library classes by their short names:

```
import   javax.swing.JButton;
...
    JButton go = new JButton("Go");
```

Fully-qualified name

import (cont'd)

- You can import names for all the classes in a package by using a wildcard .*:

```
import   java.awt.*;
import   java.awt.event.*;
import   javax.swing.*;
```

Imports all classes from **awt**, **awt.event**, and **swing** packages

- java.lang is imported automatically into all classes; defines System, Math, Object, String, and other commonly used classes.

Package

- Java is a package-centric language; for good organization and name scoping, put all classes into packages.
- A class with default access can be seen only by classes within the same package.
- If class A and class B are in different packages, and class A has default access, class B won't be able to create an instance of class A, or even declare a variable or return type of class A.

Package

- Example: class Sludge and class Goo are both in different packages

```
package cert;
public class Sludge {
  public void testIt() {
    System.out.println("sludge");
  }
}
```

```
package book;
import cert.*; // Import all classes in the cert package
class Goo {
  public static void main(String[] args) {
    Sludge o = new Sludge();
    o.testIt();
  }
}
```

Package, Directory & Class Path

- Must store **Sludge** and **Goo** classes in the **book** and **cert** directories as follows:

D:/AP

Can compile & run it from inside the parent directory/folder:

```
D:\AP> javac book/Goo.java
D:\AP> java book/Goo.java
```

```
|-- book
|    |-- Goo.java
|
|-- cert
|    |--Sludge.java
```

Need to set the class path if compile and run from outside package's drive/directory:

```
C:\> javac -classpath D:/AP D:/AP/book/Goo.java
C:\> java -classpath D:/AP book/Goo
```

Source File Declaration Rules

- A source code file can have only one public class.
- If the source file contains a public class, the filename must match the public class name.
- A file can have only one package statement, but multiple imports.
- The package statement (if any) must be the first (non-comment) line in a source file.
- The import statements (if any) must come after the package and before the class declaration.

Source File Declaration Rules (cont.)

- If there is no package statement, import statements must be the first (non-comment) statements in the source file.
- package and import statements apply to all classes in the file.
- A file can have more than one nonpublic class.
- Files with no public classes can have a name that does not match any of the classes in the file.